



Local Deployment Guide — Asset Pulse on PostgreSQL 17 (localhost:5433)

This guide walks you through running **Asset Pulse** entirely on your own computer, backed by your existing **PostgreSQL 17** installation listening on **localhost port 5433**. No Docker, no cloud accounts, no VPS — just Python, Node.js, and Postgres.

You will end up with:

- FastAPI backend running on `http://localhost:8000`
- Vite (React) frontend running on `http://localhost:5173`
- A `asset_pulse` database in PostgreSQL 17 with the schema and seed data

from `db/001_init.sql`

It is written for someone who has **never deployed an app before** — every command is spelled out. If you have done this before, jump straight to the "Cheat sheet" at the bottom.

1. Prerequisites

Install these once, then never again:

Tool	Minimum version	Notes
Python	3.10	Make sure <code>python --version</code> (or <code>python3 --version</code>) prints 3.10 or higher. Download from https://www.python.org/downloads/
Node.js	18 LTS	Download from https://nodejs.org/ . npm ships with it. Confirm with <code>node --version</code> and <code>npm --version</code> .
PostgreSQL 17	17	Already installed on <code>localhost:5433</code> per your setup. The psql command-line client must be on your PATH. On Windows, the EnterpriseDB installer adds it; on macOS Homebrew adds it via <code>brew install postgresql@17</code> ; on Linux use your distro's <code>postgresql-client-17</code> package.
Git or ZIP extractor	any	Either <code>git clone</code> the repository, or download <code>asset_pulse_local_deployment.zip</code> and extract it with the ZIP tool built into Windows/macOS/Linux.



Quick sanity check

Open a terminal (Command Prompt or PowerShell on Windows; Terminal on macOS/Linux) and run:

```
python --version          # or: python3 --version
node --version
npm --version
psql --version
```

All four should print a version number. If any one says "command not found" or "is not recognized", install or re-install that tool and reopen the terminal.

2. Get the project on disk

Option A — git clone

```
git clone <repository-url> asset_pulse
cd asset_pulse
```

Option B — extract the ZIP

1. Locate `asset_pulse_local_deployment.zip`.
2. Right-click → **Extract All** (Windows) or double-click (macOS) or

`unzip asset_pulse_local_deployment.zip` (Linux).

3. Open a terminal in the extracted folder.

The rest of this guide assumes your terminal's working directory is the **project root** (the folder that contains `backend/`, `frontend/`, `db/`, and `docs/`).

3. Create the PostgreSQL 17 database and user

PostgreSQL 17 is already running on `localhost:5433`. You now need to:

1. Create a **database** named `asset_pulse`.
2. Create a **user** (a "role") named `asset_pulse` with a password you choose.
3. Grant that user full access to the database.

Open a terminal and connect as the Postgres superuser. The default superuser is `postgres`.

```
psql -h localhost -p 5433 -U postgres
```

Enter your `postgres` password when prompted. If the prompt changes to `postgres=#` you are connected.

Now run these four SQL statements one at a time. **Replace** `change_me_to_a_strong_password` with a real password you will use only for this database:

```
CREATE DATABASE asset_pulse;
CREATE USER asset_pulse WITH PASSWORD 'change_me_to_a_strong_password';
GRANT ALL PRIVILEGES ON DATABASE asset_pulse TO asset_pulse;
```



\q

The last command (\q) exits psql.

> **PostgreSQL 15+ note.** Owning the database is no longer enough — the new > user also needs privileges on the default public schema. Connect to the > new database as the superuser and grant them: > > ``bash > psql -h localhost -p 5433 -U postgres -d asset\_pulse > ` > > `sql > GRANT ALL ON SCHEMA public TO asset\_pulse; > ALTER SCHEMA public OWNER TO asset\_pulse; > \q > ``

4. Run db/001_init.sql to create the schema and seed data

From the project root, apply the migration as the new user:

```
psql -h localhost -p 5433 -U asset_pulse -d asset_pulse -f db/001_init.sql
```

Enter the password you set in step 3. You should see a series of CREATE TABLE, CREATE INDEX, and INSERT lines with no errors.

A helper script is included for convenience:

- macOS / Linux: ./scripts/init_db.sh
- Windows: scripts\init_db.bat

Both run the same psql command using the same defaults (host localhost, port 5433, user asset_pulse, database asset_pulse). Override with environment variables (PGUSER, PGDATABASE, PGHOST, PGPORT) if you chose different names.

Verify

```
psql -h localhost -p 5433 -U asset_pulse -d asset_pulse -c "\dt"
```

You should see a table list including assets, asset_cost_profiles, price_decks, scenarios, scenario_results, cash_flows, events, and decision_matrix_runs. The price_decks table will already contain the Base 2025 seed row.

5. Configure the backend environment variables

The backend reads its database connection string from a file named backend/.env. A template is included.

```
# macOS / Linux
cp backend/.env.local.example backend/.env

# Windows (cmd.exe)
copy backend\env.local.example backend\env
```

Open backend/.env in any text editor and replace the placeholders with the database name, user, and password you created in step 3:

```
DATABASE_URL=postgresql://asset_pulse:change_me_to_a_strong_password@localhost:5433/asset_pulse
CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
```



backend/.env is excluded from git via .gitignore — it never gets committed, so your password stays on your machine.

6. Create a Python virtual environment and install requirements

A "virtual environment" is just a folder with an isolated Python interpreter and packages, so this project's dependencies do not collide with anything else on your computer.

```
cd backend
python -m venv .venv
```

> If python is not found, try python3 -m venv .venv instead. Use the > same prefix (python vs python3) for the rest of this section.

Activate the virtualenv. The activation command depends on your shell:

Shell	Command
macOS / Linux (bash, zsh)	source .venv/bin/activate
Windows — cmd.exe	.venv\Scripts\activate.bat
Windows — PowerShell	.venv\Scripts\Activate.ps1

Your prompt should now start with (.venv). Install the requirements:

```
pip install -r requirements.txt
```

This installs FastAPI, Uvicorn, SQLAlchemy, psycpg2-binary, and Pydantic.

7. Start the FastAPI backend

Still inside backend/ with the virtualenv activated:

```
./run.sh
```

...or use the cross-platform helper from the **project root** (one level up):

- macOS / Linux: ./scripts/start_backend.sh
- Windows: scripts\start_backend.bat

Either way, FastAPI binds to `http://localhost:8000`. You should see something like:

```
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:      Started reloader process
INFO:      Application startup complete.
```

Open  in your browser. The JSON response should show "database": "postgresql" (not "sqlite"). If it shows "sqlite", your DATABASE_URL is not being loaded — see Troubleshooting.

Interactive API docs are at `http://localhost:8000/docs`.

Leave this terminal running.



8. Install frontend dependencies and start the Vite dev server

Open a **second terminal window**, navigate to the project root, then:

```
cd frontend
npm install
npm run dev
```

...or use the helper from the project root:

- macOS / Linux: `./scripts/start_frontend.sh`
- Windows: `scripts\start_frontend.bat`

Vite serves the React UI on **localhost:5173**. The dev server automatically proxies `/api/*` requests to the FastAPI backend on port 8000 (configured in `frontend/vite.config.ts`), so nothing else needs to change.

9. Open the app

Browse to **`http://localhost:5173`**.

You should see the Asset Pulse terminal-style dashboard:

- The **API/DB status** indicator on the left rail should show the

PostgreSQL backend.

- Click **Assets** to load the seeded sample wells/pipelines.
- Click **Run scenario** to generate a 120-month projection.
- Explore Sensitivity, Monte Carlo, Events, Decision Matrix, and CSV

Exchange views.

That's it — you have a fully local Asset Pulse.

10. Troubleshooting

`psql: error: connection to server at "localhost" (::1), port 5433 failed: Connection refused`

PostgreSQL 17 is not listening on port 5433.

- Confirm the service is running:
- macOS (Homebrew): `brew services list`
- Linux (systemd): `systemctl status postgresql@17-main`
- Windows: open **Services** and check `postgresql-x64-17`
- Confirm the port. Look at `postgresql.conf` for `port = 5433` and

`listen_addresses = 'localhost'.`



- If your Postgres is on the default port 5432 instead, update

backend/.env accordingly.

password authentication failed for user "asset_pulse"

The password in DATABASE_URL does not match the one you set for the database user.

- Reset the password as the superuser:

```
psql -h localhost -p 5433 -U postgres -c "ALTER USER asset_pulse WITH PASSWORD 'new_password';"
```

- Update backend/.env to match. Restart the backend.

database "asset_pulse" does not exist

You skipped creating the database, or named it differently.

```
psql -h localhost -p 5433 -U postgres -c "CREATE DATABASE asset_pulse;"
```

Then re-run `psql -h localhost -p 5433 -U asset_pulse -d asset_pulse -f db/001_init.sql`.

The frontend shows CORS errors or 404s on /api/...

- Make sure the backend terminal is still running on port 8000.
- The Vite dev server **must** be running on port 5173 (the default). The

proxy in frontend/vite.config.ts only kicks in for the dev server.

- CORS_ORIGINS in backend/.env should include ``http://localhost:5173``.

Restart the backend after editing .env.

npm install fails with EACCES, EPERM, or network errors

- On Windows, run the terminal **as Administrator** once for the first

install, or use a Node.js installation in your user profile (e.g. via <https://nodejs.org/> — not a system-wide one that requires admin).

- Behind a corporate proxy, configure `npm config set proxy ...` and

```
npm config set https-proxy ....
```

- Delete frontend/node_modules and frontend/package-lock.json and try

`npm install` again only as a last resort — the lockfile is the source of truth and should normally be kept.

psycopg2 build fails on pip install (compiler errors, pg_config not found)

The requirements file pins psycopg2-binary, which ships precompiled wheels for the major OS/CPU combinations and does not need a compiler.

If you still see compiler errors, you are probably on an unusual platform (Apple Silicon M1/M2/M3 with an old pip, Linux ARM, FreeBSD). Fixes, in order:

1. Upgrade pip first: `pip install --upgrade pip setuptools wheel`, then



re-run `pip install -r requirements.txt`.

2. If you actually want the source build, install the Postgres development

headers (postgresql-server-dev-17 on Debian/Ubuntu, `brew install libpq && brew link --force libpq` on macOS) and try `pip install psycopg2==2.9.9` instead of the binary.

`/api/health` **says** `"database": "sqlite"`

Your `DATABASE_URL` is not being read.

- Check `backend/.env` exists and has no leading/trailing quotes around

the value.

- The `start_backend.sh / .bat` helpers source the file automatically.

If you launched `uvicorn` manually, export the variable yourself:

```
export $(grep -v '^#' backend/.env | xargs)          # macOS / Linux
uvicorn app.main:app --reload --port 8000
```

- Restart the server after any change to `.env`.

Backend imports work but API returns 500 on first request

The schema may not have been applied. Re-run step 4 (`psql ... -f db/001_init.sql`) and check `\dt` lists all the tables.

11. Optional: production-like local run

By default `npm run dev` ships an unminified bundle and serves through the Vite dev server. To approximate what a deployed build would look like:

```
cd frontend
npm run build
npm run preview          # serves dist/ on http://localhost:4173 by default
```

`npm run preview` does **not** proxy `/api/*` to the backend the way `npm run dev` does. For the preview to talk to your backend you have two options:

1. **Set an API base URL at build time.** Vite inlines variables prefixed

with `VITE_` into the bundle:

```
VITE_API_BASE_URL=http://localhost:8000 npm run build
npm run preview
```

2. **Loosen CORS** in `backend/.env` to include the preview origin

(`http://localhost:4173`) and restart the backend.

The FastAPI backend continues to run exactly as in step 7 — the only thing that changes is which frontend you open in the browser.



Cheat sheet

```
# One-time setup
psql -h localhost -p 5433 -U postgres -c "CREATE DATABASE asset_pulse;"
psql -h localhost -p 5433 -U postgres \
  -c "CREATE USER asset_pulse WITH PASSWORD 'change_me';"
psql -h localhost -p 5433 -U postgres \
  -c "GRANT ALL PRIVILEGES ON DATABASE asset_pulse TO asset_pulse;"
psql -h localhost -p 5433 -U postgres -d asset_pulse \
  -c "GRANT ALL ON SCHEMA public TO asset_pulse;"
psql -h localhost -p 5433 -U asset_pulse -d asset_pulse -f db/001_init.sql

cp backend/.env.local.example backend/.env      # then edit DATABASE_URL

cd backend && python -m venv .venv && source .venv/bin/activate \
  && pip install -r requirements.txt && cd ..
cd frontend && npm install && cd ..

# Every-day run (two terminals)
./scripts/start_backend.sh      # http://localhost:8000
./scripts/start_frontend.sh     # http://localhost:5173
```

Windows users: replace `./scripts/*.sh` with `scripts\*.bat`.